Lista de exercícios

Aula 07 — Listas dinâmicas encadeadas circulares

> **Entrega:** Parta da biblioteca ListaDinEncadCirc.h e ListaDinEncadCirc.c construída na Aula 07, disponível em github.com/Ensino-UFPB/exemplosed1.
>
> Escreva cada função dentro de ListaDinEncadCirc.c e declare o seu protótipo em ListaDinEncadCirc.h. Veja a observação abaixo: nenhuma destas funções compila fora do .c.
>
> Entregue os dois arquivos da biblioteca alterados e um main.c com os testes que você usou.
>
> **Sugestões (opcional):**
>
> Todo código deve compilar sem avisos com: gcc -Wall -Wextra -std=c11 ListaDinEncadCirc.c main.c -o prog
>
> Nas funções que alocam ou liberam memória, verifique o resultado com: valgrind --leak-check=full ./prog

> **Por que dentro do .c:** Todas as funções pedidas precisam percorrer a lista nó a nó, acessando os campos dados e prox de struct elemento. Esse tipo é definido dentro de ListaDinEncadCirc.c justamente para ficar oculto de quem usa a biblioteca, e o apelido Elem não existe fora dele. Uma função escrita em main.c não compila: o compilador acusa tipo incompleto.
>
> Essa é a consequência prática do tipo opaco discutido na aula de TAD. Quem usa a biblioteca enxerga apenas Lista* e as funções declaradas no .h; quem a estende trabalha por dentro.

odas operam sobre a lista dinâmica encadeada circular estudada na Aula 07, e nenhuma delas pode ser resolvida comparando o campo prox com NULL.

Questão 1 — Impressão da lista

Implemente a função imprime_lista, que percorre a lista exatamente uma volta e imprime a matrícula de cada aluno, separadas por um espaço, encerrando com uma quebra de linha. Se a lista for inválida ou estiver vazia, a função não imprime nada.

Assinatura:

```
void imprime_lista(Lista* li);
```

Exemplo:

```
Lista com 12, 16, 23 e 40:
Saída:  12 16 23 40
```

> **Restrições:** A função percorre a lista uma única vez: nenhum elemento pode ser impresso duas vezes.
>
> Nenhum nó é alocado ou liberado.

> **Sugestão:** Um laço while(no != (*li)) não imprime elemento algum, porque a condição já é falsa na primeira avaliação. Determine qual estrutura de repetição resolve isso.

Questão 2 — Maior matrícula da lista

Implemente a função maior_matricula, que escreve em *mat a maior matrícula presente na lista e devolve 1. Se a lista for inválida ou estiver vazia, a função devolve 0 e não altera *mat.

Assinatura:

```
int maior_matricula(Lista* li, int *mat);
```

Exemplo:

```
Lista com 12, 16, 23 e 40:
*mat recebe 40 e a função devolve 1.
```

> **Restrições:** Uma única volta na lista.
> Não é permitido usar vetor auxiliar nem ordenar a lista.

> **Sugestão:** Inicialize o maior com a matrícula do primeiro elemento, e não com zero: matrículas são positivas, mas iniciar com um valor arbitrário acopla a função a uma suposição sobre os dados.

Questão 3 — Identificação e correção de defeitos

A função abaixo deveria contar quantos alunos da lista têm média das três notas maior ou igual a 7,0. Ela compila sem nenhum aviso, inclusive com -Wall -Wextra, e mesmo assim está errada. Identifique os dois defeitos, explique o efeito de cada um durante a execução e escreva a versão corrigida.

Código com defeito:

```
int conta_aprovados(Lista* li){
    int cont = 0;
    Elem *no = *li;
    while(no != NULL){
        float media = (no->dados.n1 + no->dados.n2 + no->dados.n3) / 3;
        if(media >= 7.0)
            cont++;
        no = no->prox;
    }
    return cont;
}
```

Assinatura:

```
int conta_aprovados(Lista* li);
```

> **Restrições:** A explicação deve dizer o que acontece na execução, e não apenas nomear o erro.
> A versão corrigida mantém a mesma assinatura.

> **Sugestão:** Pergunte-se qual é o valor do campo prox do último elemento de uma lista circular, e o que a função faz quando recebe uma lista recém-criada.

Questão 4 — Busca cíclica por posição

Implemente a função busca_ciclica, que copia para *al o k-ésimo elemento contado a partir do início da lista, percorrendo-a ciclicamente. O valor de k pode ser maior que o número de elementos: ao passar do último, a contagem continua no primeiro. A posição k = 1 corresponde ao próprio início. Valores de k menores ou iguais a zero são inválidos. A função devolve 1 em caso de sucesso e 0 caso contrário.

Assinatura:

```
int busca_ciclica(Lista* li, int k, struct aluno *al);
```

Exemplo:

```
Lista com 12, 16, 23 e 40:
k = 1  ->  12        k = 4  ->  40
k = 5  ->  12        k = 7  ->  23
```

> **Restrições:** O número de avanços de ponteiro não pode crescer com k: uma chamada com k = 1000 em uma lista de 4 elementos deve custar o mesmo que uma chamada com k = 4.
>
> É permitido usar tamanho_lista.

> **Sugestão:** O resto da divisão inteira resolve o problema. Determine por qual valor dividir e quantos avanços são necessários para chegar à posição k = 1.

Questão 5 — Remoção por posição

Implemente a função remove_lista_pos, que remove o elemento que ocupa a posição pos da lista, contada a partir de 1 no início. A função devolve 1 em caso de sucesso e 0 se a lista for inválida, estiver vazia, ou se pos estiver fora do intervalo válido. A memória do elemento removido deve ser liberada.

Assinatura:

```
int remove_lista_pos(Lista* li, int pos);
```

Exemplo:

```
Lista com 12, 16, 23 e 40:
remove_lista_pos(li, 2)  ->  12, 23, 40
remove_lista_pos(li, 1)  ->  23, 40   (o início passa a ser 23)
```

> **Restrições:** Três casos de borda precisam ser tratados: remoção da primeira posição, remoção da última e remoção do único elemento da lista.
>
> Diferentemente da busca cíclica, aqui pos maior que o número de elementos é inválido e deve devolver 0.
>
> É permitido reutilizar remove_lista_inicio.

> **Sugestão:** A remoção da primeira posição altera o conteúdo de (*li) e já foi implementada na aula. Para as demais, o que a função precisa localizar é o antecessor do elemento a remover.

Questão 6 — Rotação da lista

Implemente a função rotaciona_lista, que avança o início da lista em k posições. Após a chamada, o elemento que ocupava a posição k + 1 passa a ser o primeiro, e os elementos que o precediam passam para o fim da sequência. A função devolve 1 em caso de sucesso e 0 se a lista for inválida, estiver vazia ou se k for negativo. Responda também, por escrito, às duas perguntas da caixa de restrições.

Assinatura:

```
int rotaciona_lista(Lista* li, int k);
```

Exemplo:

```
Lista com 12, 16, 23 e 40:
rotaciona_lista(li, 1)  ->  16, 23, 40, 12
rotaciona_lista(li, 4)  ->  lista inalterada (volta completa)
```

> **Restrições:** Nenhum nó pode ser alocado, liberado ou ter o campo prox alterado. A rotação é uma mudança de ponto de entrada, e não de estrutura.
>
> Pergunta 1: qual é o custo da função, e por que ela não precisa ser O(N) para k pequeno?
>
> Pergunta 2: o que seria necessário para obter o mesmo efeito em uma lista dinâmica encadeada não circular, e qual seria o custo?

> **Sugestão:** Se nenhum campo prox muda e nenhum nó é alocado, resta uma única coisa na estrutura que pode ser alterada. Identifique qual é.

Questão 7 — Concatenação de duas listas

Implemente a função concatena_listas, que anexa todos os elementos de li2 ao final de li1, preservando a ordem das duas e mantendo a circularidade do resultado. Ao final, li2 deve estar vazia, mas continuar sendo uma lista válida, pronta para receber novas inserções. A função devolve 1 em caso de sucesso e 0 se algum dos ponteiros for inválido.

Assinatura:

```
int concatena_listas(Lista* li1, Lista* li2);
```

Exemplo:

```
li1 com 1, 2  e  li2 com 3, 4:
concatena_listas(li1, li2)  ->  li1 fica 1, 2, 3, 4  e  li2 fica vazia
```

> **Restrições:** Nenhum nó pode ser alocado, copiado ou liberado: a concatenação é feita apenas religando ponteiros.
>
> Os casos em que li1 está vazia e em que li2 está vazia precisam ser tratados.
>
> Responda por escrito: qual seria o custo desta mesma operação na representação que guarda o final da lista em (*li), apresentada no fim da aula?

> **Sugestão:** Uma lista circular tem um único ponto de corte natural: a ligação entre o último e o primeiro elemento. Concatenar duas listas circulares é desfazer esse corte nas duas e refazê-lo cruzado.

Questão 8 — Problema de Josephus

N pessoas estão dispostas em círculo e são eliminadas de k em k, sempre no mesmo sentido: conta-se a partir do início da lista e a k-ésima pessoa é eliminada; a contagem recomeça na pessoa seguinte à eliminada, e o processo se repete até restar uma única pessoa. Implemente a função josephus, que escreve em *sobrevivente a matrícula da última pessoa restante, libera a memória de todos os nós e deixa a lista vazia. A função devolve 1 em caso de sucesso e 0 se a lista for inválida, estiver vazia ou se k não for positivo.

Assinatura:

```
int josephus(Lista* li, int k, int *sobrevivente);
```

Exemplo:

```
Lista com as matrículas 1 a 7 e k = 3:
eliminados, nesta ordem:  3, 6, 2, 7, 5, 1
sobrevivente:  4
```

> **Restrições:** Não é permitido usar vetor auxiliar nem marcar elementos como eliminados: cada pessoa eliminada tem o seu nó efetivamente removido da lista e liberado com free.
>
> O caso de uma lista com um único elemento precisa funcionar sem entrar no laço de eliminação.

> **Sugestão:** Mantenha dois ponteiros, o corrente e o seu antecessor, exatamente como nas remoções estudadas na aula. O antecessor inicial é o último elemento da lista. A condição de término do processo é a mesma que a aula usa para detectar lista de um elemento só.

Desafio opcional — a representação que guarda o final

O fechamento da aula mostrou que armazenar em (*li) o endereço do último elemento, em vez do primeiro, reduz a O(1) a inserção nas duas extremidades e a remoção do início. Reimplemente as três funções nessa representação e escreva um programa que comprove o comportamento.

- Reescreva insere_lista_inicio, insere_lista_final e remove_lista_inicio supondo que (*li) guarda o final da lista.
- Adapte tamanho_lista e imprime_lista, que continuam O(N), à nova posição armazenada.
- Escreva um main() que monte uma lista de dez elementos, imprima o conteúdo após cada operação e confirme que a ordem é a mesma obtida com a implementação original.
- Justifique, em até cinco linhas, por que a remoção do final permanece O(N) mesmo nessa representação.